



Vidyavardhini's

College of Engineering & Technology

Vasai Road (W)

**Department of Artificial Intelligence & Data Science
Engineering**

Laboratory Manual Student Copy

Semester	III	Class	S.E.
Course Code	2483115		
Course Name	Analysis of Algorithms Lab		



Vidyavardhini's College of Engineering & Technology

Vision

To be a premier institution of technical education; always aiming at becoming a valuable resource for industry and society.

Mission

- To provide technologically inspiring environment for learning.
- To promote creativity, innovation and professional activities.
- To inculcate ethical and moral values.
- To cater personal, professional and societal needs through quality education.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Department Vision:

To foster proficient artificial intelligence and data science professionals, making remarkable contributions to industry and society.

Department Mission:

- To encourage innovation and creativity with rational thinking for solving the challenges in emerging areas.
- To inculcate standard industrial practices and security norms while dealing with Data.
- To develop sustainable Artificial Intelligence systems for the benefit of various sectors.

Program Specific Outcomes (PSOs):

PSO1: Analyse the current trends in the field of Artificial Intelligence & Data Science and convey their finding by presenting / publishing at a national / international forums.

PSO2: Design and develop Artificial Intelligence & Data Science based solutions and applications for the problems in the different domains catering to industry and society.



Program Outcomes (POs):

Engineering Graduates will be able to:

- **PO1. Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
- **PO2. Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
- **PO3. Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
- **PO4. Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
- **PO5. Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
- **PO6. The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
- **PO7. Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
- **PO8. Individual and teamwork:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
- **PO9. Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
- **PO10. Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
- **PO11. Life-long learning:** Recognize the need for and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Course Objective

1	To introduce the methods of designing and analysing algorithms
2	Design and implement efficient algorithms for a specified application
3	Strengthen the ability to identify and apply the suitable algorithm for the given real-world problem.
4	Analyse worst-case running time of algorithms and understand fundamental algorithmic problems.

Course Outcomes

At the end of the course student will be able to:		Action verbs	Bloom's Level
2483115.1	Analyze time complexity of sorting algorithms	Analyze	Apply (Level 3)
2483115.2	Analyze the complexity of problems solved using divide and conquer approaches	Analyze	Apply (Level 3)
2483115.3	Implement greedy algorithms for solving Dijkstras, Minimum spanning tree & fractional knapsack.	Implement	Apply (Level 3)
2483115.4	Implement dynamic programming algorithm for All pair shortest path and 0/1 knapsack	Apply	Apply (Level 3)
2483115.5	Implement backtracking and branch and bound for 15 puzzle, N queen and sum of subset problem	Apply	Apply (Level 3)
2483115.6	Analyze the performance of string-matching techniques	Analyze	Apply (Level 3)



Mapping of Experiments with Course Outcomes

Sr. No	Title	24831 15.1	2483 115.2	24831 15.3	24831 15.4	24831 15.5	2483 115.6
1.	To implement Insertion Sort and Comparative analysis for large values of 'n'.	3	-	-	-	-	-
2.	To implement Selection Sort and Comparative analysis for large values of 'n'	3	-	-	-	-	-
3.	To implement Binary Search for 'n' number and perform analysis using DAC Technique.	-	3	-	-	-	-
4.	To implement Merge Sort and Comparative analysis for large values of 'n' using DAC technique.	-	3	-	-	-	-
5.	To implement Fractional Knap Sack using Greedy Method.	-	-	3	-	-	-
6.	To implement Job Sequencing with deadlines.	-	-	3	-	-	-
7.	To implement All Pairs Shortest Path Algorithm using Dynamic Method (Floyd Warshall).	-	-	-	3	-	-
8.	To implement Sum of Subset Algorithm using Backtracking Method	-	-	-	3	-	-
9.	To implement N-queen Algorithm using Backtracking method	-	-	-	-	3	-
10.	Implement the Naïve string-matching algorithm and analyse its complexity.	-	-	-	-	-	3

Enter correlation level 1, 2 or 3 as defined below

1: Slight (Low)

2: Moderate (Medium)

3: Substantial (High)

If there is no correlation put “—”.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

List of Experiments

Sr. No	Name of Experiments	DOP	DOC	Marks	Sign
1.	To implement Insertion Sort and Comparative analysis for large values of 'n'.				
2.	To implement Selection Sort and Comparative analysis for large values of 'n'				
3.	To implement Binary Search for 'n' number and perform analysis using DAC Technique.				
4.	To implement Merge Sort and Comparative analysis for large values of 'n' using DAC technique..				
5.	To implement Fractional Knap Sack using Greedy Method.				
6.	To implement Job Sequencing with deadlines.				
7.	To implement All Pairs Shortest Path Algorithm using Dynamic Method (Floyd Warshall).				
8.	To implement Sum of Subset Algorithm using Backtracking Method				
9.	To implement N-queen Algorithm using Backtracking method				
10.	Implement the Naïve string-matching algorithm and analyse its complexity.				
Assignments					
1.	Assignment 1: Sorting Algorithms				
2.	Assignment 2: Divide and Conquer Approach				



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

3.	Assignment 3 : Greedy Method Approach				
4.	Assignment 4 : Dynamic Programming Approach				
5.	Assignment 5: Backtracking and Branch and Bound				
6.	Assignment 6: String Matching Algorithms				
Formative Assessment					
1	TH- Quiz 1: Sorting Algorithms				
2	TH- Quiz 2: Divide and Conquer Approach				
3	TH- Quiz 3: Greedy Method Approach				
4	TH- Quiz 4: Dynamic Programming Approach				
5	TH- Quiz 5: Backtracking and Branch and Bound				
6	TH - Quiz 6: String Matching Algorithms				
7	PR - Quiz 1: Insertion and Selection Sort				
8	PR- Quiz 2: Divide and Conquer Approach				
9	PR- Quiz 3: Fractional Knapsack and Job Sequencing, MST				
10	PR- Quiz 4: 0/1 Knapsack, TSP, All Pair Shortest Path algorithm				
11	PR- Quiz 5: N-Queen and 15 Puzzle Problem				
12	TH - Quiz 6: Naïve String , KMP and Rabin Karp				

D.O.P: Date of performance

D.O.C : Date of correction



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Experiment No.1	
Insertion Sort	
Name: Anshraj Saha	Roll No:47
Date of Performance:	
Date of Submission:	



Experiment No. 1

Title: Insertion Sort

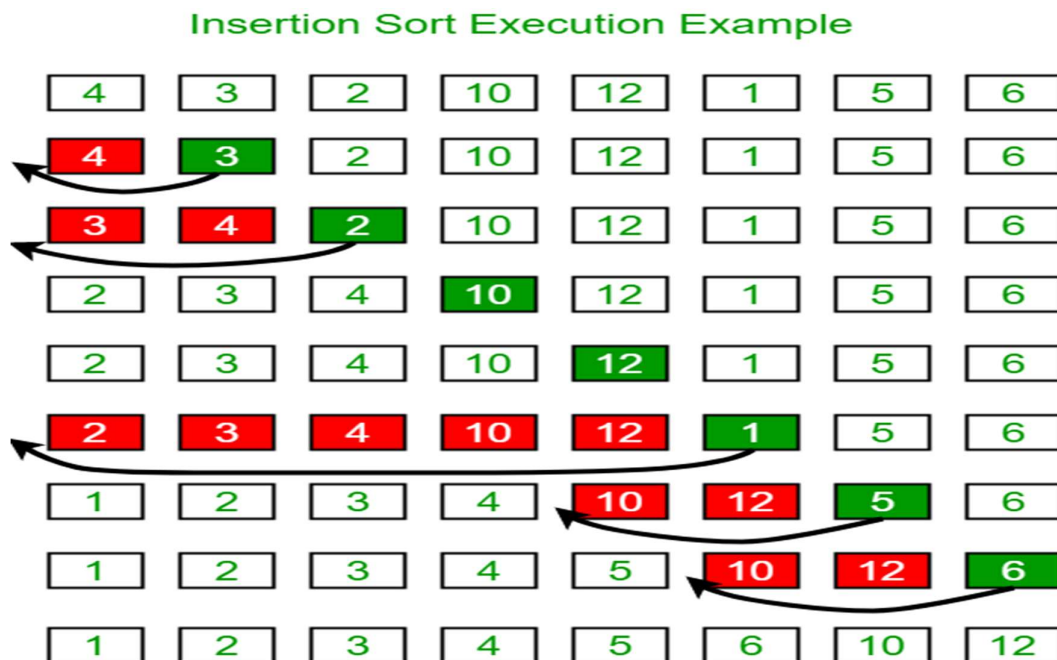
Aim: To implement Selection Comparative analysis for large values of 'n'

Objective: To introduce the methods of designing and analysing algorithms

Theory:

Insertion sort is a simple sorting algorithm that works similar to the way you sort playing cards in your hands. The array is virtually split into a sorted and an unsorted part. Values from the unsorted part are picked and placed at the correct position in the sorted part.

Example:



Algorithm and Complexity:



INSERTION-SORT (<i>A</i>)	<i>cost</i>	<i>times</i>
1 for <i>j</i> = 2 to <i>A.length</i>	c_1	n
2 <i>key</i> = <i>A</i> [<i>j</i>]	c_2	$n - 1$
3 // Insert <i>A</i> [<i>j</i>] into the sorted sequence <i>A</i> [1.. <i>j</i> - 1].	0	$n - 1$
4 <i>i</i> = <i>j</i> - 1	c_4	$n - 1$
5 while <i>i</i> > 0 and <i>A</i> [<i>i</i>] > <i>key</i>	c_5	$\sum_{j=2}^n t_j$
6 <i>A</i> [<i>i</i> + 1] = <i>A</i> [<i>i</i>]	c_6	$\sum_{j=2}^n (t_j - 1)$
7 <i>i</i> = <i>i</i> - 1	c_7	$\sum_{j=2}^n (t_j - 1)$
8 <i>A</i> [<i>i</i> + 1] = <i>key</i>	c_8	$n - 1$

Implementation:

Code :

```
#include <stdio.h>
```

```
void insertionSort(int arr[], int n){
    int i, key, j;
    for(i=1;i<n;i++){
        key = arr[i];
        j = i-1;
        while(j>=0 && arr[j]>key){
            arr[j+1] = arr[j];
            j--;
        }
        arr[j+1] = key;
    }
}
```

```
int main(){
    int arr[] = {5, 2, 9, 1, 5};
    int n = sizeof(arr)/sizeof(arr[0]);
    insertionSort(arr,n);

    printf("Sorted array: ");
    for(int i=0;i<n;i++) printf("%d ", arr[i]);
    return 0;
}
```

Output :

Sorted array: 1 2 5 5 9



Conclusion:

Insertion Sort performs efficiently for small datasets but becomes slow for large values of n due to its $O(n^2)$ complexity.

The comparative analysis shows that its execution time grows quadratically, making it unsuitable for large inputs.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Experiment No.2	
Selection Sort	
Name: Anshraj Saha	Roll No:47
Date of Performance:	
Date of Submission:	



Experiment No. 2

Title: Selection Sort

Aim: To implement Selection Comparative analysis for large values of 'n'

Objective: To introduce the methods of designing and analyzing algorithms

Theory:

Selection sort is a sorting algorithm, specifically an in-place comparison sort. Selection sort is noted for its simplicity, and it has performance advantages over more complicated algorithms in certain situations, particularly where auxiliary memory is limited.

The algorithm divides the input list into two parts: the sub list of items already sorted, which is built up from left to right at the front (left) of the list, and the sub list of items remaining to be sorted that occupy the rest of the list. Initially, the sorted sub list is empty and the unsorted sub list is the entire input list. The algorithm proceeds by finding the smallest (or largest, depending on sorting order) element in the unsorted sub list, exchanging it with the leftmost unsorted element (putting it in sorted order), and moving the sub list boundaries one element to the right.

Example:

`arr[] = 64 25 12 22 11`

`// Find the minimum element in arr[0...4] // and place it at beginning`

`11 25 12 22 64`

`// Find the minimum element in arr[1...4] // and place it at beginning of arr[1...4]`

`11 12 25 22 64`

`// Find the minimum element in arr[2...4] // and place it at beginning of arr[2...4]`

`11 12 22 25 64`

`// Find the minimum element in arr[3...4] // and place it at beginning of arr[3...4]`

`11 12 22 25 64`



Algorithm and Complexity:

Alg.: SELECTION-SORT(A)		
	cost	Times
$n \leftarrow \text{length}[A]$	c_1	1
for $j \leftarrow 1$ to $n - 1$	c_2	$n-1$
do $\text{smallest} \leftarrow j$	c_3	$n-1$
for $i \leftarrow j + 1$ to n	c_4	$\sum_{j=1}^{n-1} (n-j+1)$
$\approx n/2$ comparisons, do if $A[i] < A[\text{smallest}]$	c_5	$\sum_{j=1}^{n-1} (n-j)$
then $\text{smallest} \leftarrow i$	c_6	$\sum_{j=1}^{n-1} (n-j)$
$\approx n$ exchanges, exchange $A[j] \leftrightarrow A[\text{smallest}]$	c_7	$n-1$

Implementation:

Code :

```
#include <stdio.h>
```

```
void selectionSort(int arr[], int n){
    int i,j,min_idx,temp;
    for(i=0;i<n-1;i++){
        min_idx = i;
        for(j=i+1;j<n;j++){
            if(arr[j]<arr[min_idx])
                min_idx = j;
        }
        temp = arr[min_idx];
        arr[min_idx] = arr[i];
        arr[i] = temp;
    }
}
```

```
int main(){
    int arr[] = {64,25,12,22,11};
    int n = sizeof(arr)/sizeof(arr[0]);
    selectionSort(arr,n);

    printf("Sorted array: ");
    for(int i=0;i<n;i++) printf("%d ", arr[i]);
    return 0;
}
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Output :

Sorted array: 11 12 22 25 64

Conclusion:

Selection Sort provides consistent performance regardless of input order but remains inefficient for large n .

Its $O(n^2)$ time complexity makes it inferior to more advanced sorting algorithms for larger datasets.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Experiment No. 3
Binary Search Algorithm
Date of Performance:
Date of Submission:



Experiment No. 3

Title: Binary Search Algorithm

Aim: To study and implement Binary Search Algorithm

Objective: To introduce Divide and Conquer based algorithms

Theory:

Search a sorted array by repeatedly dividing the search interval in half. Begin with an interval covering the whole array. If the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half. Otherwise, narrow it to the upper half. Repeatedly check until the value is found or the interval is empty

- Binary search is efficient than linear search. For binary search, the array must be sorted, which is not required in case of linear search.
- It is divide and conquer based search technique.
- In each step the algorithms divides the list into two halves and check if the element to be searched is on upper or lower half the array
- If the element is found, algorithm returns.



The idea of binary search is to use the information that the array is sorted and reduce the time complexity to $O(\log n)$.



- ☐ Compare x with the middle element.
- ☐ If x matches with the middle element, we return the mid index.
- ☐ Else If x is greater than the mid element, then x can only lie in the right half subarray after the mid element. So we recur for the right half.
- ☐ Else (x is smaller) recur for the left half.
- ☐ Binary Search reduces search space by half in every iterations. In a linear search, search space was reduced by one only.
- ☐ n=elements in the array
- ☐ Binary Search would hit the bottom very quickly.

	Linear Search	Binary Search
2 nd iteration	n-1	n/2
3 rd iteration	n-2	n/4

Example:



Algorithm `BINARY_SEARCH(A, key)`

// Description: Perform BS on array A

// I/P : array A of size n & key element to be searched.

// O/P : Success/failure.

$low \leftarrow 1$

$high \leftarrow n$

while $low < high$ do

$mid \leftarrow (low + high) / 2$

 if $A[mid] == key$ then

 return mid

 else if $A[mid] < key$ then

$low \leftarrow mid + 1$

 else

$high \leftarrow mid - 1$

end

return 0

$A = \{11, 22, 33, 44, 55, 66, 77, 88\}$

$key = 33$

$low = 1$

$high = 8$

$mid = (1 + 8) / 2 = 4$

$A[4] == 33 \times$

$A[4] < 33 \times$

44

$high = 4 - 1$

$high = 3$

$\{11, 22, 33\}$

1 2 3

$low = 1$

$high = 3$

$mid = (1 + 3) / 2 = 2$

$A[2] == 33 \times$

$22 < 33$

$low = 3$

$\{33\}$ $mid = (3 + 3) / 2 = 3$

$A[3] = 33$

$A[mid] = 33$

$key = A[3]$



Algorithm and Complexity:

The binary search

- Algorithm 3: the binary search algorithm

Procedure binary search (x : integer, $a_1, a_2, \dots, a_n$: increasing integers)

$i := 1$ { i is left endpoint of search interval }

$j := n$ { j is right endpoint of search interval }

While $i < j$

begin

$m := \lfloor (i + j) / 2 \rfloor$

if $x > a_m$ **then** $i := m + 1$

else $j := m$

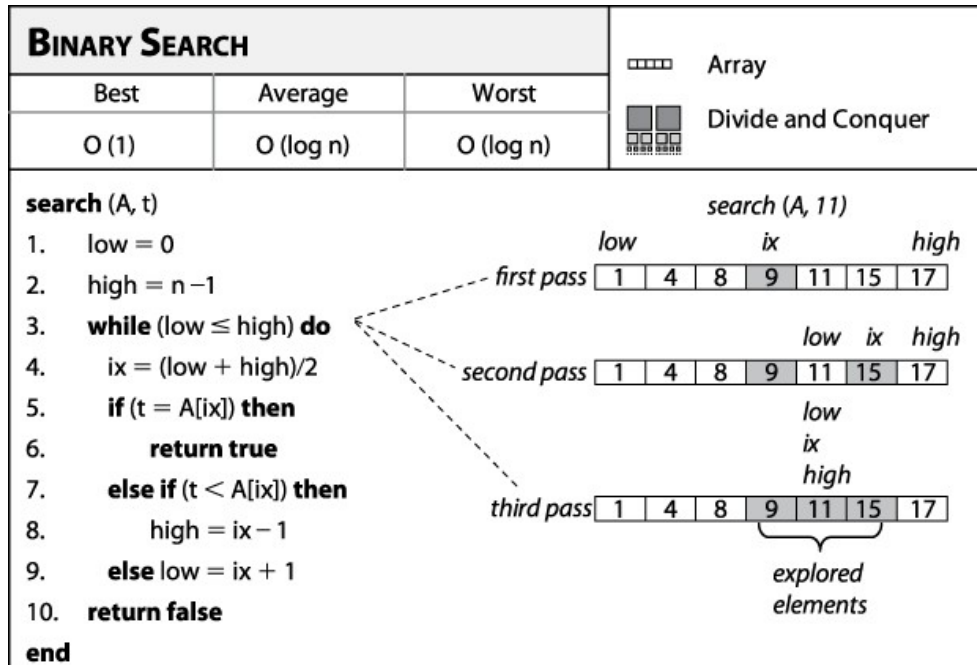
end

If $x = a_i$ **then** $location := i$

else $location := 0$

{ $location$ is the subscript of the term equal to x , or 0 if x is not found }

2





Best Case:

Key is first compared with the middle element of the array.

The key is in the middle position of the array, the algorithm does only one comparison, irrespective of the size of the array.

$$T(n)=1$$

Worst Case:

In each iteration search space of BS is reduced by half, Maximum $\log n$ (base 2) array divisions are possible.

Recurrence relation is

$$T(n)=T(n/2) + 1$$

Running Time is $O(\log n)$.

Average Case:

Key element neither is in the middle nor at the leaf level of the search tree.

It does half of the $\log n$ (base 2).

Base case= $O(1)$

Average and worst case= $O(\log n)$

Implementation:

Code :

```
#include <stdio.h>
```

```
int binarySearch(int arr[], int n, int x){
    int l = 0, r = n-1;
    while(l <= r){
        int mid = l + (r-l)/2;
        if(arr[mid] == x)
            return mid;    // found
        else if(arr[mid] < x)
            l = mid + 1;    // search right half
        else
            r = mid - 1;    // search left half
    }
    return -1;            // not found
}
```

```
int main(){
    int arr[] = {1,3,5,7,9,11,13};
    int n = sizeof(arr)/sizeof(arr[0]);
    int x = 7;

    int result = binarySearch(arr,n,x);
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
if(result != -1)
    printf("Element %d found at index %d\n", x, result);
else
    printf("Element not found\n");

return 0;
}
```

Output :

Element 7 found at index 3

Conclusion:

Binary Search efficiently reduces the problem size by half in each step using the Divide and Conquer approach.

Its logarithmic time complexity $O(\log n)$ demonstrates excellent scalability for large, sorted datasets.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Experiment No. 4
Merge Sort
Date of Performance:
Date of Submission:



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Experiment No. 4

Title: Merge Sort

Aim: To implement Merge Sort and Comparative analysis for large values of 'n' using DAC technique.

Objective: To introduce the methods of designing and analysing algorithms.

Theory:

1. Merge Sort is a sorting algorithm. The algorithm divides the array into two halves, recursively sorts them, and finally merges the two sorted halves.
2. The merge() function is used for merging two halves. The merge(arr, l, m, r) is a key process that assumes that arr[l..m] and arr[m+1..r] are sorted and merges the two sorted sub-arrays into one.
3. The array is recursively divided into two halves till the size becomes 1. Once the size becomes 1, the merge processes come into action and start merging arrays back till the complete array is merged.
4. **MergeSort(arr[], l, r)** If $r > l$
 1. Find the middle point to divide the array into two halves: $\text{middle } m = l + (r-l)/2$
 2. Call mergeSort for first half: Call mergeSort(arr, l, m)
 3. Call mergeSort for second half: Call mergeSort(arr, m+1, r)
 4. Merge the two halves sorted in step 2 and 3: Call merge(arr, l, m, r)
5. Divide: Split the array into two halves.

Conquer: Recursively sort both halves.

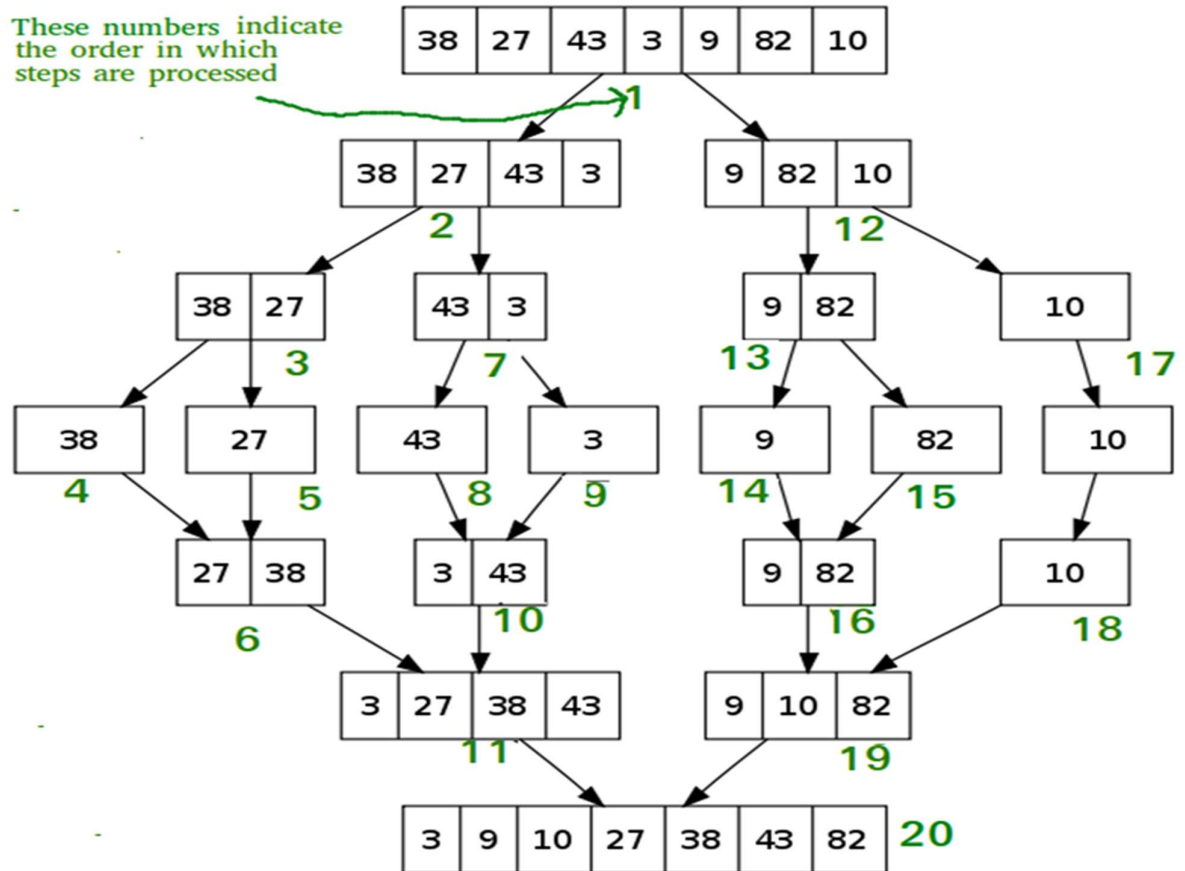
Combine: Merge the two sorted halves into one sorted array.

Time Complexity

- **Best / Average / Worst:** $O(n \log n)$



Example:



Implementation:

Code :

```
#include <stdio.h>
```

```
void merge(int arr[], int l, int m, int r){
    int n1 = m-l+1, n2 = r-m;
    int L[n1], R[n2];

    for(int i=0;i<n1;i++) L[i] = arr[l+i];
    for(int i=0;i<n2;i++) R[i] = arr[m+1+i];

    int i=0, j=0, k=l;
    while(i<n1 && j<n2){
```



```
        if(L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while(i<n1) arr[k++] = L[i++];
    while(j<n2) arr[k++] = R[j++];
}

void mergeSort(int arr[], int l, int r){
    if(l < r){
        int m = l + (r-l)/2;    // Divide
        mergeSort(arr, l, m);    // Conquer left
        mergeSort(arr, m+1, r); // Conquer right
        merge(arr, l, m, r);    // Combine
    }
}

int main(){
    int arr[] = {38,27,43,3,9,82,10};
    int n = sizeof(arr)/sizeof(arr[0]);

    printf("Original array: ");
    for(int i=0;i<n;i++) printf("%d ", arr[i]);
    printf("\n");
    mergeSort(arr, 0, n-1);
    printf("Sorted array: ");
    for(int i=0;i<n;i++) printf("%d ", arr[i]);
    printf("\n");

    return 0;
}
```

Output :

Original array: 38 27 43 3 9 82 10

Sorted array: 3 9 10 27 38 43 82

Conclusion: Merge Sort, based on Divide and Conquer, shows consistent $O(n \log n)$ performance even for large inputs.

Its stability and predictable efficiency make it ideal for handling large datasets compared to quadratic sorts.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Experiment No. 5	
Fractional Knapsack using Greedy Method	
Name: Anshraj Saha	Roll No:47
Date of Performance:	
Date of Submission:	



Experiment No. 5

Title: Fraction Knapsack

Aim: To study and implement Fraction Knapsack Algorithm

Objective: To introduce Greedy based algorithms

Theory:

Greedy method or technique is used to solve Optimization problems. A solution that can be maximized or minimized is called Optimal Solution.

The knapsack problem or rucksack problem is a problem in combinatorial optimization: Given a set of items, each with a mass and a value, determine the number of each item to include in a collection so that the total weight is less than or equal to a given limit and the total value is as large as possible. It derives its name from the problem faced by someone who is constrained by a fixed size knapsack and must fill it with the most valuable items. The most common problem being solved is the 0-1 knapsack problem, which restricts the number x_i of copies of each kind of item to zero or one.

In Knapsack problem we are given: 1) n objects 2) Knapsack with capacity m , 3) An object i is associated with profit W_i , 4) An object i is associated with profit P_i , 5) when an object i is placed in knapsack we get profit $P_i X_i$.

Here objects can be broken into pieces (X_i Values) The Objective of Knapsack problem is to maximize the profit.

Example:



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

In this version of Knapsack problem, items can be broken into smaller pieces. So, the thief

greedy-fractional-knapsack ($w[1..n], p[1..n], W$)

```

for i=1 to n
  do  $x[i] = 0$ 
  weight = 0
  for i=1 to n
    if  $\text{weight} + w[i] \leq W$  then
       $x[i] = 1$ 
       $\text{weight} = \text{weight} + w[i]$ 
    else
       $x[i] = (W - \text{weight}) / w[i]$ 
       $\text{weight} = W$ 
      break
  return x
  
```

Example:

$W = 60$

Item	A	B	C	D
profit	280	100	120	120
weight	40	10	20	20
Ratio ($\frac{p_i}{w_i}$)	7	10	6	5

provided items are not sorted based on $\frac{p_i}{w_i}$.

sorted

Item	B	A	C	D
profit	100	280	120	120
weight	10	40	20	20
Ratio ($\frac{p_i}{w_i}$)	10	7	6	5

Handwritten calculations:

$i=1 \rightarrow B$
 $0 + 10 \leq 60$
 $x[B] = 1$
 $wt = 10$

$i=2 \rightarrow A$
 $10 + 40 \leq 60$
 $x[A] = 1$
 $wt = 50$

$i=3 \rightarrow C$
 $(60 - 50) / 20$
 $x[C] = 10 / 20 = 1/2$
 $wt = 60$

$x = [A, B, 1/2 C]$

Total profit is
 $100 + 280 + 120 * (10/20)$
 $380 + 60 = 440$

Total wt
 $10 + 40 + 20 * (10/20)$
 $= 60$

may take only a fraction x_i of i^{th} item.

$$0 \leq x_i \leq 1$$

The i^{th} item contributes the weight $x_i \cdot w_i$ to the total weight in the knapsack and profit $x_i \cdot p_i$ to the total profit.



Algorithm:

Hence, the objective of this algorithm is to

$$\text{maximize } \sum_{i=1}^n (x_i \cdot p_i)$$

subject to constraint,

$$\sum_{i=1}^n (x_i \cdot w_i) \leq W$$

It is clear that an optimal solution must fill the knapsack exactly, otherwise we could add a fraction of one of the remaining items and increase the overall profit.

Thus, an optimal solution can be obtained by

$$\sum_{i=1}^n (x_i \cdot w_i) = W$$

In this context, first we need to sort those items according to the value of $\frac{p_i}{w_i}$, so that $\frac{p_i+1}{w_i+1} \leq$

$\frac{p_i}{w_i}$. Here, x is an array to store the fraction of items.

Algorithm: Greedy-Fractional-Knapsack ($w[1..n]$, $p[1..n]$, W)

```
for i = 1 to n
    do x[i] = 0
weight = 0
for i = 1 to n
    if weight + w[i] ≤ W then
        x[i] = 1
        weight = weight + w[i]
    else
        x[i] = (W - weight) / w[i]
        weight = W
        break
return x
```




Implementation:

Code :

```
#include <stdio.h>
```

```
double fractionalKnapsack(int n, double w[], double v[], double W, double x[]) {  
    double weight = 0, totalValue = 0;  
    for(int i=0;i<n;i++) x[i] = 0;  
    for(int i=0;i<n;i++){  
        if(weight + w[i] <= W){  
            x[i] = 1;  
            weight += w[i];  
            totalValue += v[i];  
        } else {  
            x[i] = (W - weight)/w[i];  
            totalValue += x[i]*v[i];  
            break;  
        }  
    }  
    return totalValue;  
}
```

```
int main() {  
    int n;  
    printf("Enter number of items: ");  
    scanf("%d", &n);  
  
    double w[n], v[n], x[n], W, totalValue;  
  
    printf("Enter weights: ");  
    for(int i=0;i<n;i++) scanf("%lf",&w[i]);  
    printf("Enter values: ");  
    for(int i=0;i<n;i++) scanf("%lf",&v[i]);  
    printf("Enter knapsack capacity: ");  
    scanf("%lf",&W);  
    totalValue = fractionalKnapsack(n, w, v, W, x);  
  
    printf("Fractions taken:\n");  
    for(int i=0;i<n;i++) printf("Item %d: %.2f\n", i+1, x[i]);  
}
```




Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
printf("Total value = %.2f\n", totalValue);  
  
return 0;  
}
```

Input:

Enter number of items : 3

Enter weights: 10 20 30

Enter values: 60 100 120

Enter knapsack capacity: 50

Output :

Fractions taken:

Item 1: 1.00

Item 2: 1.00

Item 3: 0.67

Total value = 240.00

Conclusion: The Fractional Knapsack algorithm provides an optimal solution by selecting items based on maximum value-to-weight ratio.

Its $O(n \log n)$ efficiency and greedy nature make it fast and reliable for real-time resource allocation problems.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Experiment No. 6	
Job Sequencing with Deadlines	
Name: Anshraj Saha	Roll No:47
Date of Performance:	
Date of Submission:	



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Experiment No. 6

Title: Job Sequencing with deadlines.

Aim:. To schedule jobs in such a way that the total profit is maximized, while ensuring that each job is completed within its deadline.

Objective:

To find an optimal sequence of jobs that can be completed within the given deadlines.

To maximize total profit earned from completed jobs.

To utilize available time slots efficiently without overlapping jobs.

To apply a greedy approach to select the most profitable jobs first and assign them the latest possible slot before their deadline.

Theory:

1. Sort all jobs in decreasing order of profit.

(Jobs with higher profit come first.)

2. Find the maximum deadline among all jobs (let it be max_d).

3. Create an array 'slot[1...max_d]' to represent available time slots.

Initialize all slots as empty.

4. For each job in the sorted list:

a. Find a free slot from job's deadline down to 1

(i.e., try to place the job in the latest available slot before its deadline)

b. If a free slot is found, assign the job to that slot.

Implementation:

Job	Deadline	Profit
A	2	100
B	1	19
C	2	27
D	1	25
E	3	15



Step 1: Sort by profit (descending)

A(100), C(27), D(25), B(19), E(15)

Step 2: Max deadline = 3 $\rightarrow$ slots = [_, _, _]

Step 3: Assign jobs

Job	Deadline	Assigned Slot	Slots Status
A	2	Slot 2	[_, A, _]
C	2	Slot 1	[C, A, _]
D	1	— (Slot 1 filled)	[C, A, _]
B	1	— (Slot 1 filled)	[C, A, _]
E	3	Slot 3	[C, A, E]

Scheduled Jobs: C $\rightarrow$ A $\rightarrow$ E

Total Profit: 27 + 100 + 15 = 142

◆ Time Complexity

Sorting: $O(n \log n)$

Slot checking: $O(n \times \text{maxDeadline})$

Overall: $O(n^2)$ (can be optimized)

Implementation:

Code :

```
#include <stdio.h>
```

```
int jobsequence(int d[], int n, int j[]) {  
    int i, q, r, k;  
    k = 0;  
    for(i = 1; i <= n; i++) {  
        r = i - 1;
```



```
while(r >= 0 && d[j[r]] > d[i]) r--;  
    if(r < 0 || d[j[r]] <= d[i]) {  
        for(q = k; q > r + 1; q--) j[q] = j[q - 1];  
        j[r + 1] = i;  
        k++;  
    }  
}  
return k;  
}  
  
int main() {  
    int d[6], p[6], j[6], k, i, n = 5;  
  
    for(i = 1; i <= n; i++) scanf("%d", &d[i]);  
    for(i = 1; i <= n; i++) scanf("%d", &p[i]);  
  
    for(i = 1; i <= n; i++) j[i] = i;  
  
    k = jobsequence(d, n, j);  
  
    for(i = 0; i < k; i++)  
        printf("Job %d with profit %d and deadline %d\n", j[i], p[j[i]], d[j[i]]);  
  
    return 0;  
}
```

Input :

2
1
2
1
3
100
19
27
25
15



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Output :

Job 2 with profit 19 and deadline 1

Job 1 with profit 100 and deadline 2

Job 3 with profit 27 and deadline 2

Job 5 with profit 15 and deadline 3

Conclusion:

The Job Sequencing algorithm maximizes profit by scheduling jobs in order of descending profit and feasible deadlines.

Its greedy approach ensures near-optimal scheduling with $O(n \log n)$ efficiency.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Experiment No. 7	
All Pairs Shortest Path Algorithm	
Name: Anshraj Saha	Roll No:47
Date of Performance:	
Date of Submission:	



Experiment No. 7

Title: All Pair Shortest path, Dynamic Programming: Floyd Warshall

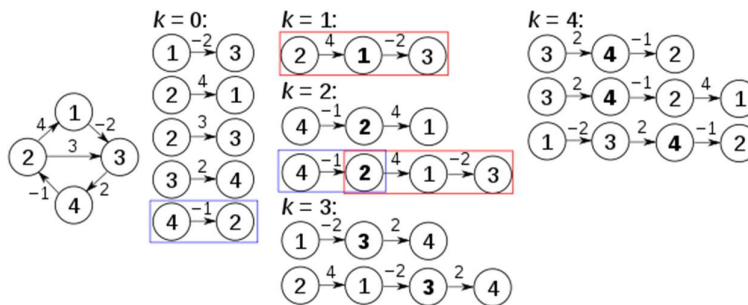
Aim: To study and implement All Pair Shortest path, Dynamic Programming: Floyd Warshall.

Objective: To introduce Dynamic Programming Approaches

Theory:

the Floyd–Warshall algorithm is an algorithm for finding shortest paths in a directed weighted graph with positive or negative edge weights (but with no negative cycles). A single execution of the algorithm will find the lengths (summed weights) of shortest paths between all pairs of vertices. Although it does not return details of the paths themselves, it is possible to reconstruct the paths with simple modifications to the algorithm. Versions of the algorithm can also be used for finding the transitive closure of a relation 'R', or (in connection with the Schulze voting system) widest paths between all pairs of vertices in a weighted graph.

Example:



Algorithm:

FLOYD-WARSHALL(W)

1. $n \leftarrow \text{rows}[W]$
2. $D^{(0)} \leftarrow W$
3. **for** $k \leftarrow 1$ **to** n
4. **do for** $i \leftarrow 1$ **to** n
5. **do for** $j \leftarrow 1$ **to** n
6. $d_{ij}^{(k)} \leftarrow \min(d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)})$
7. **return** $D^{(n)}$



Implementation:

Code :

```
#include <stdio.h>
#include <stdlib.h>

void floydWarshall(int **graph, int n)
{
    int i, j, k;
    for (k = 0; k < n; k++)
    {
        for (i = 0; i < n; i++)
        {
            for (j = 0; j < n; j++)
            {
                if (graph[i][j] > graph[i][k] + graph[k][j])
                    graph[i][j] = graph[i][k] + graph[k][j];
            }
        }
    }
}

int main(void)
{
    int n, i, j;
    printf("Enter the number of vertices: ");
    scanf("%d", &n);
    int **graph = (int **)malloc(n * sizeof(int *));
    for (i = 0; i < n; i++)
    {
        graph[i] = (int *)malloc(n * sizeof(int));
    }
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
            if (i == j)
                graph[i][j] = 0;
            else
                graph[i][j] = 100; // Large value representing infinity
        }
    }
    printf("Enter the adjacency matrix (use 100 for no edge):\n");
    for (i = 0; i < n; i++)
    {

```



```
for (j = 0; j < n; j++)
{
    printf("[%d][%d]: ", i, j);
    scanf("%d", &graph[i][j]);
}
}
printf("\nOriginal Graph:\n");
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        printf("%3d ", graph[i][j]);
    }
    printf("\n");
}
floydWarshall(graph, n);
printf("\nShortest Path Matrix:\n");
for (i = 0; i < n; i++)
{
    for (j = 0; j < n; j++)
    {
        printf("%3d ", graph[i][j]);
    }
    printf("\n");
}
for (i = 0; i < n; i++)
    free(graph[i]);
free(graph);

return 0;
}
```

Output :

```
Enter the number of vertices: 3
Enter the adjacency matrix (use 100 for no edge):
[0][0]: 56
[0][1]: 1
[0][2]: 11
[1][0]: 5
[1][1]: 45
[1][2]: 1
[2][0]: 45
[2][1]: 1
[2][2]: 0
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Original Graph:

```
56  1 11
 5 45  1
45  1  0
```

Shortest Path Matrix:

```
6  1  2
5  2  1
6  1  0
```

Conclusion: The Floyd–Warshall algorithm effectively computes shortest paths between all vertex pairs using dynamic programming.

Though its $O(n^3)$ complexity limits scalability, it guarantees optimal results for dense graphs.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Experiment No. 8	
Sum of Subset Algorithm	
Name: Anshraj Saha	Roll No:47
Date of Performance:	
Date of Submission:	



Experiment No: 8

Title: Sum of Subset

Aim: To study and implement Sum of Subset problem

Objective: To introduce Backtracking methods

Theory:

Backtracking is finding the solution of a problem whereby the solution depends on the previous steps taken. For example, in a maze problem, the solution depends on all the steps you take one-by-one. If any of those steps is wrong, then it will not lead us to the solution. In a maze problem, we first choose a path and continue moving along it. But once we understand that the particular path is incorrect, then we just come back and change it. This is what backtracking basically is.

In backtracking, we first take a step and then we see if this step taken is correct or not i.e., whether it will give a correct answer or not. And if it doesn't, then we just come back and change our first step. In general, this is accomplished by recursion. Thus, in backtracking, we first start with a partial sub-solution of the problem (which may or may not lead us to the solution) and then check if we can proceed further with this sub-solution or not. If not, then we just come back and change it.

Thus, the general steps of backtracking are:

- Start with a sub-solution
- check if this sub-solution will lead to the solution or not
- If not, then come back and change the sub-solution and continue again.

The subset sum problem is a classic optimization problem that involves finding a subset of a given set of positive integers whose sum matches a given target value. More formally, given a set of non-negative integers and a target sum, we aim to determine whether there exists a subset of the integers whose sum equals the target.

Let's consider an example to better understand the problem. Suppose we have a set of integers [1, 4, 6, 8, 2] and a target sum of 9. We need to determine whether there exists a subset within



the given set whose sum equals the target, in this case, 9. In this example, the subset [1, 8] satisfies the condition, as their sum is indeed 9.

Solving Subset Sum with Backtracking

To solve the subset, sum problem using backtracking, we will follow a recursive approach. Here's an outline of the algorithm:

1. Sort the given set of integers in non-decreasing order.
2. Start with an empty subset and initialize the current sum as 0.
3. Iterate through each integer in the set:
 - Include the current integer in the subset.
 - Increment the current sum by the value of the current integer.
 - Recursively call the algorithm with the updated subset and current sum.
 - If the current sum equals the target sum, we have found a valid subset.
 - Backtrack by excluding the current integer from the subset.
 - Decrement the current sum by the value of the current integer.

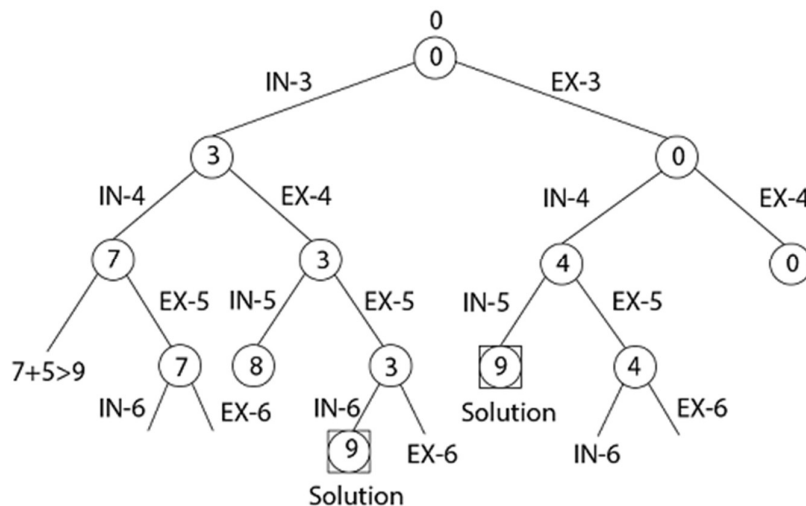
4.If we have exhausted all the integers and none of the subsets sum up to the target, we conclude that there is no valid subset.

Example:

Given a set $S = (3, 4, 5, 6)$ and $X = 9$. Obtain the subset sum using Backtracking approach.

Initially $S = (3, 4, 5, 6)$ and $X = 9$.

$S' = (\emptyset)$





Implementation:

Code :

```
#include <stdio.h>
#include <stdbool.h>

void printSubset(int subset[], int size) {
    printf("Subset: { ");
    for (int i = 0; i < size; i++) {
        printf("%d ", subset[i]);
    }
    printf("}\n");
}

void sumOfSubsetsUtil(int weights[], int targetSum, int n, int subset[], int subsetSize, int
sum, int index) {
    if (sum == targetSum) {
        printSubset(subset, subsetSize);
        return;
    }

    for (int i = index; i < n; i++) {
        if (sum + weights[i] <= targetSum) {
            subset[subsetSize] = weights[i];
            sumOfSubsetsUtil(weights, targetSum, n, subset, subsetSize + 1, sum +
weights[i], i + 1);
        }
    }
}

void sumOfSubsets(int weights[], int targetSum, int n) {
    int subset[n];
    sumOfSubsetsUtil(weights, targetSum, n, subset, 0, 0, 0);
}

int main() {
    int n, targetSum;

    printf("Enter the number of elements: ");
    scanf("%d", &n);

    int weights[n];

    printf("Enter the elements: ");
    for (int i = 0; i < n; i++) {
```



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

```
scanf("%d",&weights[i]);  
} printf("Enter the target sum: ");  
scanf("%d",&targetSum);  
  
sumOfSubsets(weights, targetSum, n);  
return 0;  
}
```

Output :

```
Enter the number of elements: 5  
Enter the elements: 4  
8  
12  
15  
20  
Enter the target sum: 35  
Subset: { 8 12 15 }  
Subset: { 15 20 }
```

Conclusion: The Sum of Subset algorithm systematically explores all combinations to find subsets matching the target sum.

Backtracking efficiently prunes the search space, though exponential complexity remains for larger sets.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Experiment No. 9	
N-Queen Algorithm	
Name: Anshraj Saha	Roll No:47
Date of Performance:	
Date of Submission:	



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Experiment No. 9

Title: N-Queen

Aim: To study and implement N-Queen

Objective: To introduce Backtracking and Branch-Bound methods

Theory:

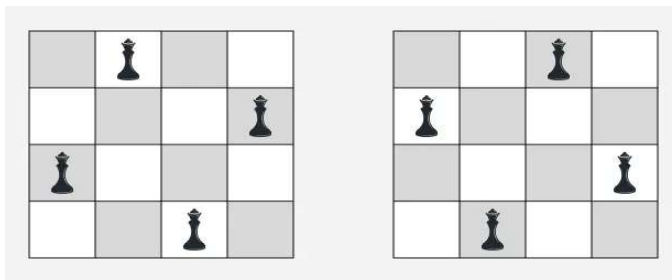
Given an integer n , place n queens on an $n \times n$ chessboard such that no two queens attack each other. A queen can attack another queen if they are placed in the same row, the same column, or on the same diagonal. For example, following is a solution for 4 Queen problem.

Example:

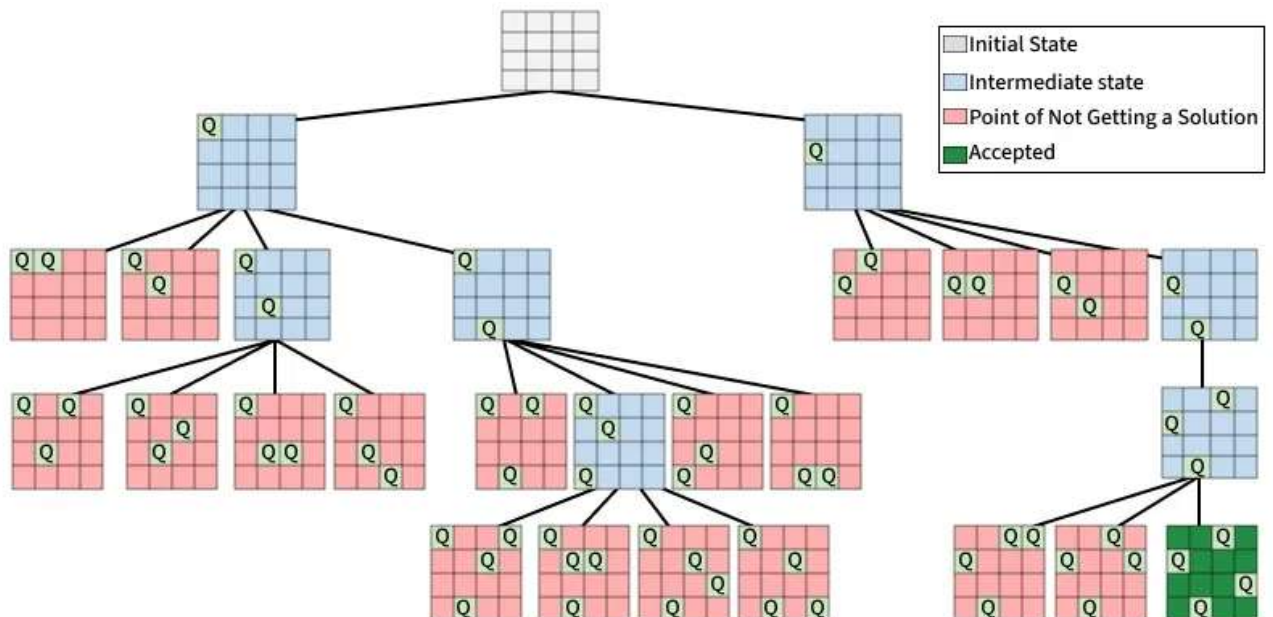
Input: $n = 4$

Output: $[[2, 4, 1, 3], [3, 1, 4, 2]]$

Explanation: Below is Solution for 4 queen problem



State Space Tree:





Implementation:

Code :

```
#include<stdio.h>
#include<math.h>

int board[20],count;
int main()
{
    int n,i,j;
    void queen(int row,int n);
    printf(" - N Queens Problem Using Backtracking -");
    printf("\n\nEnter number of Queens:");
    scanf("%d",&n);
    queen(1,n);
    return 0;
}

void print(int n)
{
    int i,j;
    printf("\n\nSolution %d:\n\n",++count);
    for(i=1;i<=n;++i)
        printf("\t%d",i);
    for(i=1;i<=n;++i)
    {
        printf("\n\n%d",i);
        for(j=1;j<=n;++j) {
            if(board[i]==j)
                printf("\tQ");
            else
                printf("\t-");
        }
    }
}

int place(int row,int column)

{
    int i;
    for(i=1;i<=row-1;++i)
```



{

```
if(board[i]==column)
return 0;
else
if(abs(board[i]-column)==abs(i-row))
return 0;
}
return 1;
}
```

```
void queen(int row,int n)
{
int column;
for(column=1;column<=n;++column)
{
if(place(row,column))
{
board[row]=column;
if(row==n) print(n);
else
queen(row+1,n);
}
}
}
```

Output :

- N Queens Problem Using Backtracking -

Enter number of Queens: 4

Solution 1:

	1	2	3	4
1	-	Q	-	-
2	-	-	-	Q
3	Q	-	-	-
4	-	-	Q	-



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Solution 2:

	1	2	3	4
1	-	-	Q	-
2	Q	-	-	-
3	-	-	-	Q
4	-	Q	-	-

Conclusion: The N-Queen algorithm successfully places N queens on the board without conflict using recursive backtracking.

It demonstrates the strength of backtracking in solving complex constraint-satisfaction problems efficiently.



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Experiment No. 10	
Naïve String matching	
Name: Anshraj Saha	Roll No:47
Date of Performance:	
Date of Submission:	



Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

Experiment No. 10

Title: Naïve String matching

Aim: To study and implement Naïve string-matching Algorithm

Objective: To introduce String matching methods

Theory:

The naïve approach tests all the possible placement of Pattern P [1.....m] relative to text T [1.....n]. We try shift $s = 0, 1, \dots, n-m$, successively and for each shift s . Compare T [s+1.....s+m] to P [1.....m].

The naïve algorithm finds all valid shifts using a loop that checks the condition $P [1.....m] = T [s+1.....s+m]$ for each of the $n - m + 1$ possible value of s .

Example:

Text : A A B A A C A A D A A B A A B A

Pattern : A A B A

A	A	B	A									A	A	B	A
A	A	B	A	A	C	A	A	D	A	A	B	A	A	B	A
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
												A	A	B	A

Pattern Found at 0, 9 and 12

Algorithm:



THE NAIVE ALGORITHM

The naive algorithm finds all valid shifts using a loop that checks

the condition $P[1\dots m]=T[s+1\dots s+m]$ for each of the $n-m+1$

possible values of s . (P =pattern , T =text/string , s =shift)

NAIVE-STRING-MATCHER(T,P)

- 1) $n = T.length$
- 2) $m = P.length$
- 3) **for** $s=0$ to $n-m$
- 4) **if** $P[1\dots m]==T[s+1\dots s+m]$
- 5) **printf** "Pattern occurs with
 shift " s

Implementation:

Code :

```
#include<stdio.h>
```

```
void search(char* pat, char* txt) {  
    int M = strlen(pat);  
    int N = strlen(txt);  
  
    for (int i = 0; i <= N - M; i++) {  
        int j;  
        for (j = 0; j < M; j++) {  
            if (txt[i + j] != pat[j]) {  
                break;  
            }  
        }  
        if (j == M) {  
            printf("Pattern found at index %d\n", i);  
        }  
    }  
}
```




Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

}

```
int main() {  
    char txt1[] = "AABAACAADAABAABA";  
    char pat1[] = "AABA";  
    printf("Example 1:\n");  
    search(pat1, txt1);  
    char txt2[] = "agd";  
    char pat2[] = "g";  
    printf("\nExample 2:\n");  
    search(pat2, txt2);  
    return 0;  
}
```

Output :

Pattern found at index 0
Pattern found at index 9
Pattern found at index 13

Conclusion: The Naïve String Matching algorithm checks all possible alignments, leading to $O(m \times n)$ time complexity.

Though simple to implement, it performs poorly on large texts compared to advanced pattern-matching methods.